VIT BHOPAL UNIVERSITY

Build Your Own Project

SCIENTIFIC CALCULATOR

A Modular Command-Line Scientific Calculation Application

Name: **Ashutosh Tiwari**

Registration Number: **26BAI10540**

Technology: Python, math, Pytest, Git

Project Type: Command-Line Scientific Calculator

1. Introduction

The VITyarthi Build Your Own Project brief asks students to identify a meaningful problem, design a technical solution, implement it using course concepts, and demonstrate the result through documentation and evaluation. This project applies Python programming concepts to a menu-driven scientific calculator. The supplied calculator functionality is preserved while the implementation is organized into meaningful modules for arithmetic, scientific functions, validation, menu handling, application dispatch, and the user interface.

2. Problem Statement

Students frequently need quick calculations involving arithmetic, powers, roots, factorials, and trigonometric values. Repeating these calculations manually can be inconvenient and may lead to avoidable input or arithmetic errors. The proposed solution is a lightweight command-line scientific calculator that accepts user choices and numeric input, performs the selected operation, displays the result, and handles common invalid cases safely.

3. Objectives

- Provide a simple menu-driven interface for scientific calculations.
- Implement common arithmetic operations: addition, subtraction, multiplication, division, power, and square.
- Implement square root, factorial, sine, cosine, and tangent operations.
- Accept trigonometric angles in degrees for user-friendly input.
- Validate numeric input and handle division by zero and invalid mathematical domains.
- Demonstrate modular Python programming, exception handling, and automated testing.

4. Functional Requirements

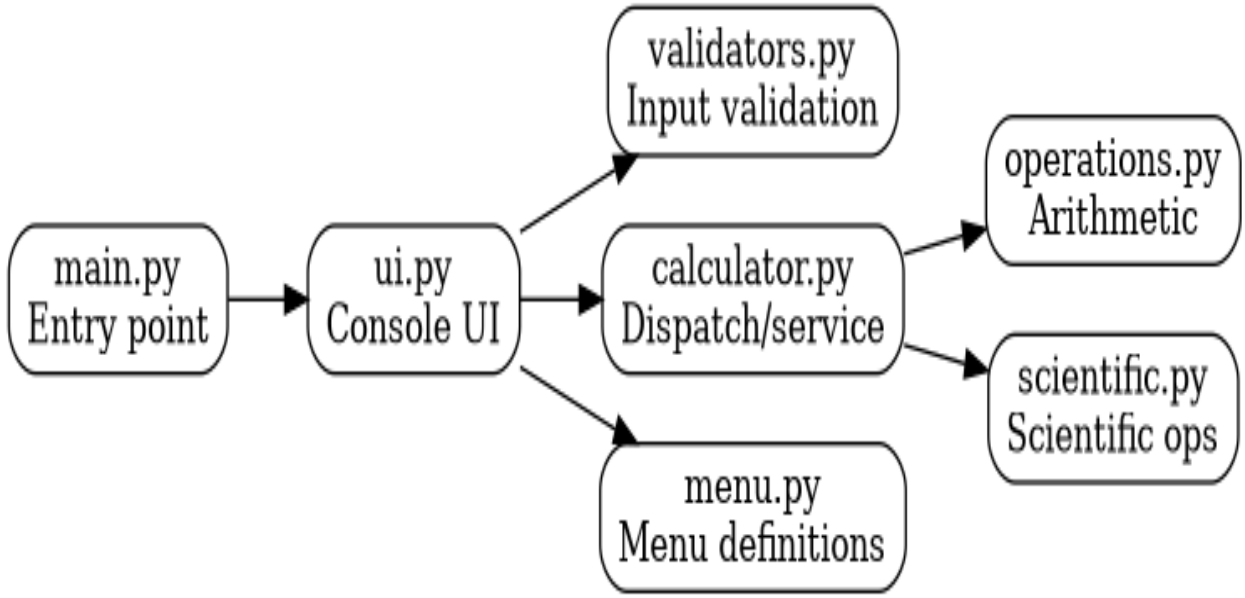
ID	Functional requirement	Input	Output
FR1	Display operation menu	None	Numbered menu
FR2	Perform basic arithmetic	Two numbers	Calculated result
FR3	Perform power and square	One/two numbers	Calculated result
FR4	Perform root and factorial	Number	Calculated result or error
FR5	Perform trigonometric operations	Angle in degrees	Sine/cosine/tangent
FR6	Validate input and errors	User text/numbers	Helpful error message
FR7	Exit application	Choice 12	Exit confirmation

5. Non-Functional Requirements

- Usability: the numbered menu and direct prompts keep interaction simple for beginners.
- Reliability: division by zero and invalid mathematical inputs are checked before returning a result.
- Maintainability: calculation logic is split into small modules with focused responsibilities.
- Performance: operations use Python standard-library functions and complete immediately for normal inputs.
- Portability: the application uses standard Python functionality and runs in a terminal on supported systems.
- Testability: arithmetic, scientific functions, dispatch, and error cases can be tested independently.

6. System Architecture

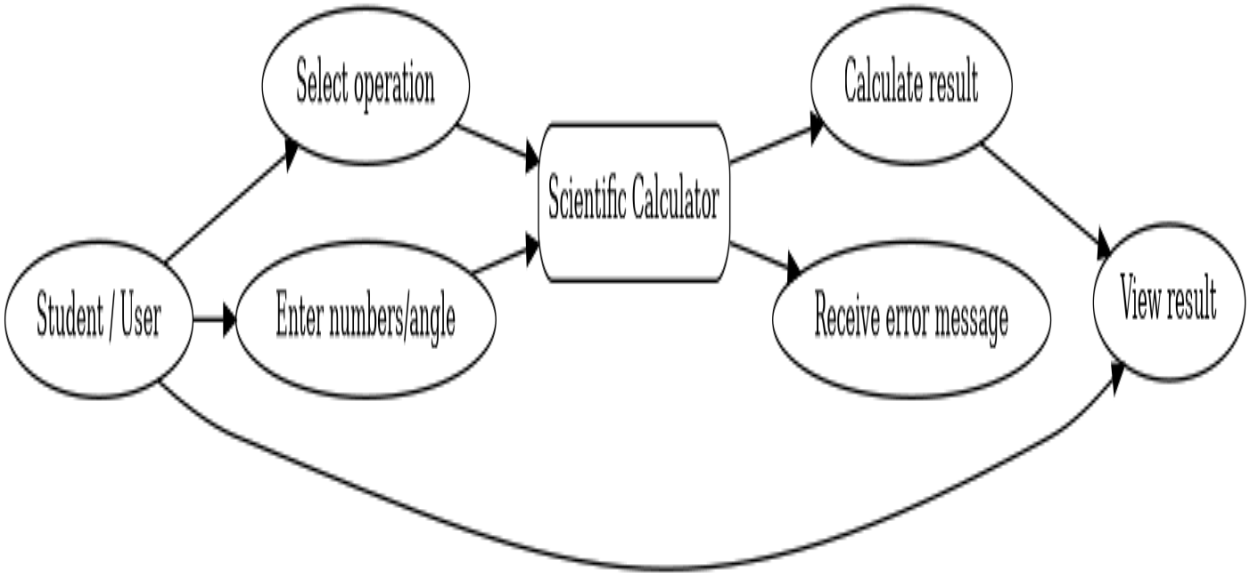
The system follows a small layered architecture. The console UI receives input and displays output; validators convert text to numeric types; the application service dispatches menu choices; operation modules perform arithmetic/scientific calculations; and the main module starts the application. No database or persistent storage is required because the calculator is stateless between runs.



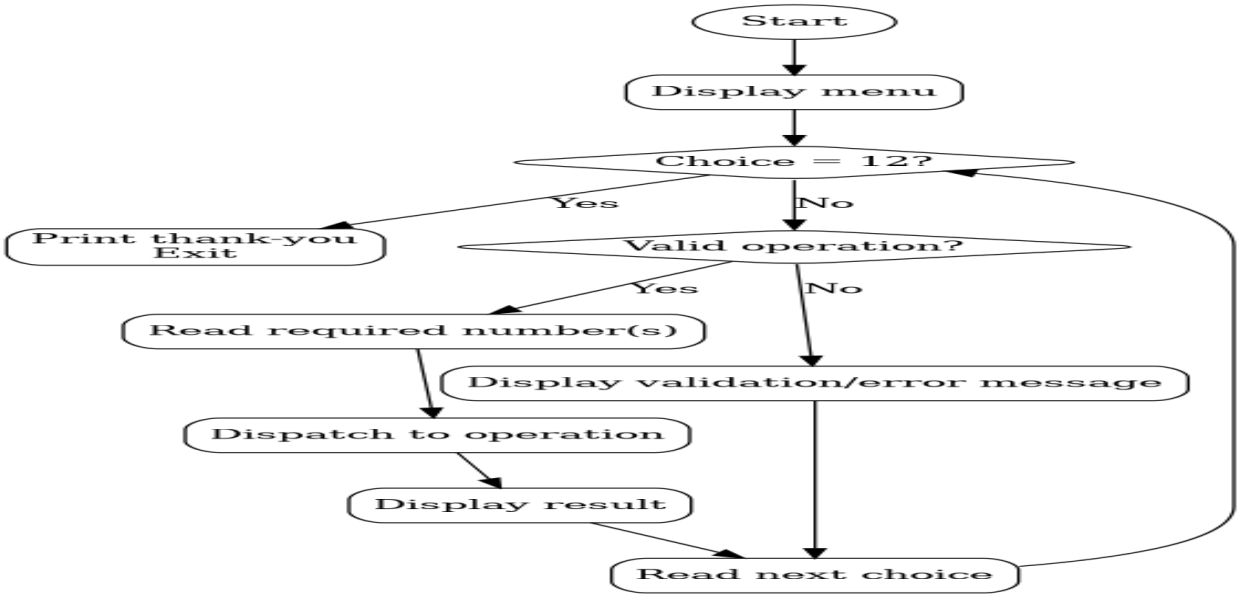
7. Design Diagrams

The diagrams document the required user interaction, workflow, code flow, module relationships, execution sequence, and storage design. Storage design explicitly records that the calculator does not use persistent storage.

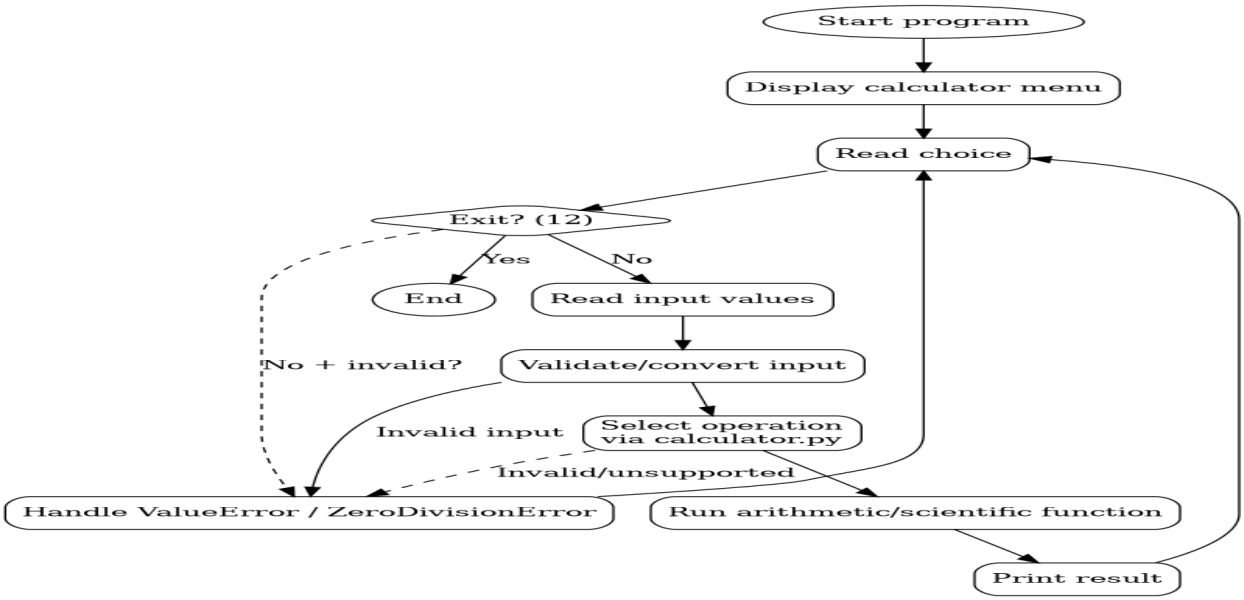
Use Case Diagram



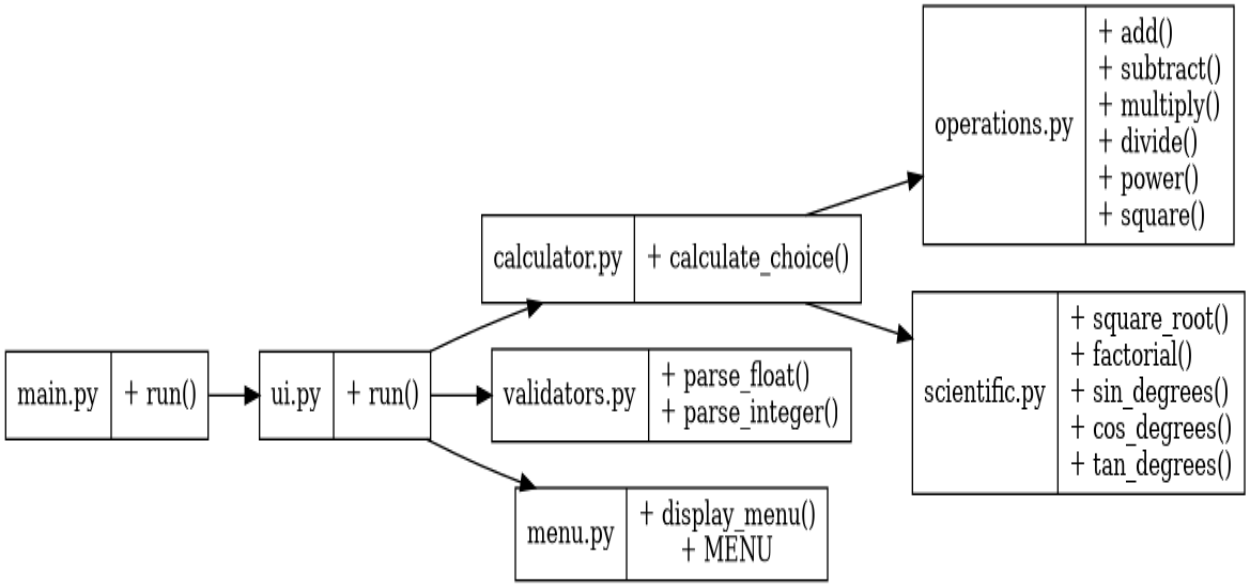
Workflow Diagram



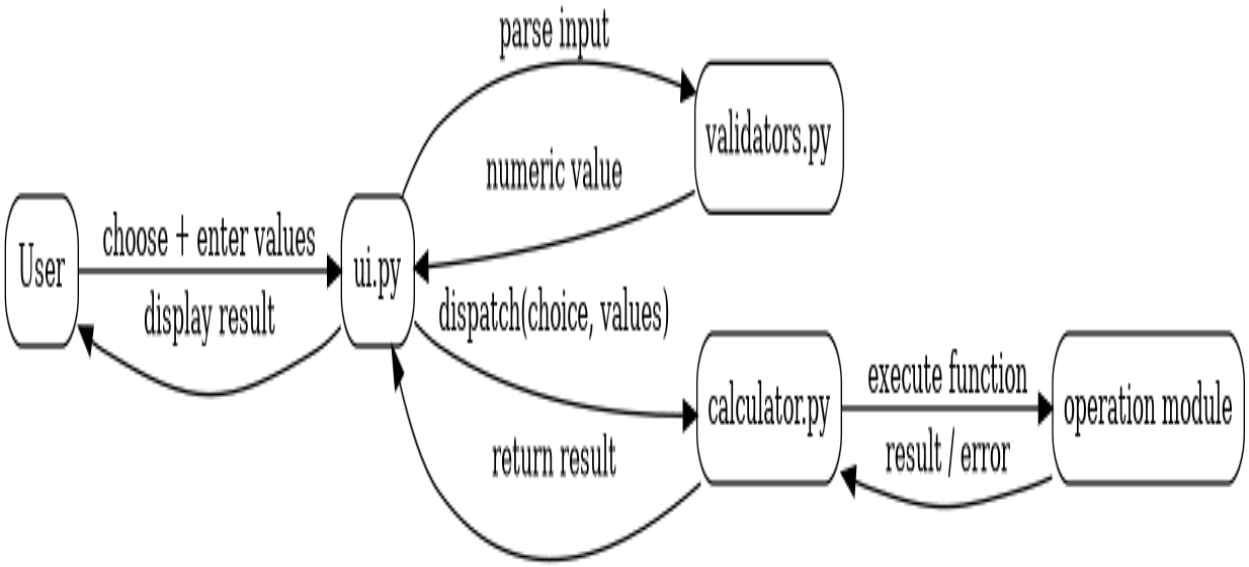
Code Flowchart


Start program
Display calculator menu
Read choice
Exit? (12)
Yes
No
End
Read input values
No + invalid?
Validate/convert input
Invalid input
Select operation via calculator.py
Invalid/unsupported
Handle ValueError / ZeroDivisionError
Run arithmetic/scientific function
Print result

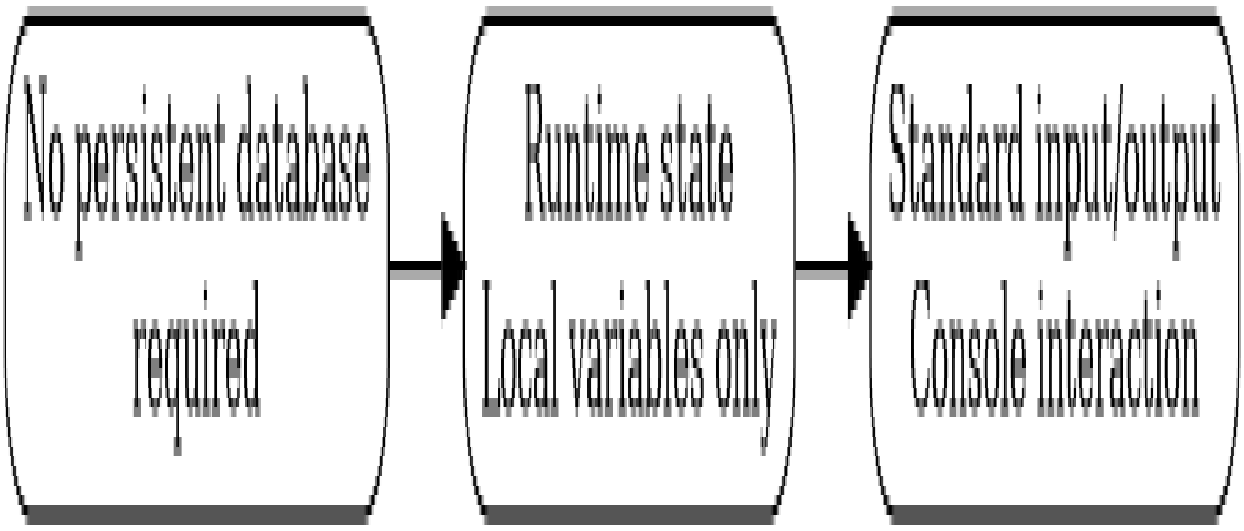
Class / Component Diagram



Sequence Diagram



Storage Design


No persistent database required
Runtime state
Local variables only
Standard input/output
Console interaction

8. Design Decisions & Rationale

- Modularization: arithmetic and scientific operations are separated so individual functions can be tested without the UI.
- Dispatcher design: calculator.py maps menu choices to operations, keeping menu interaction separate from calculation implementation.
- Degree-based trigonometry: the original functionality asks for angles in degrees, so the scientific module converts degrees to radians before calling math functions.
- Validation: numeric conversion occurs before calculations, allowing invalid text to be handled without terminating the program.
- Standard library: the math module is sufficient for the required scientific functions, avoiding unnecessary dependencies.

9. Implementation Details

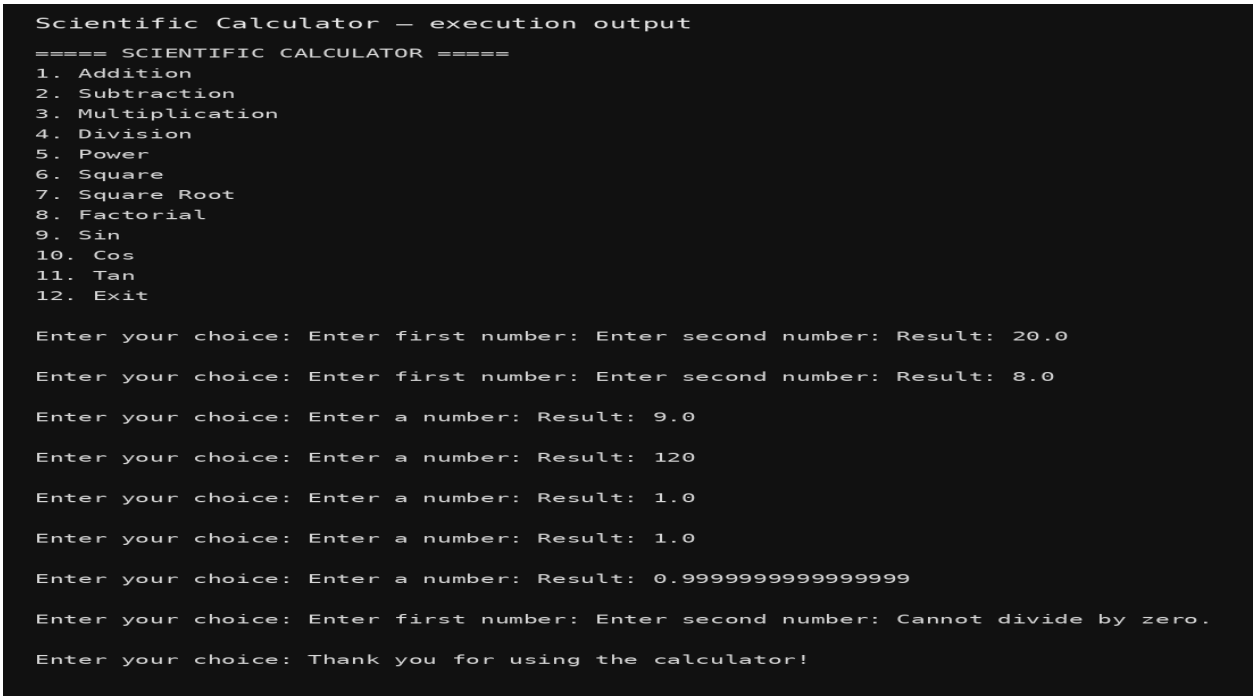
The source code is organized into seven meaningful application modules plus tests. main.py is the entry point. ui.py controls the console loop and prompts. menu.py defines and displays the options. validators.py converts input text to floats or integers. calculator.py dispatches a selected operation. operations.py contains arithmetic functions. scientific.py contains root, factorial, and trigonometric functions. The implementation uses exceptions for invalid values and division by zero.

10. Project Structure

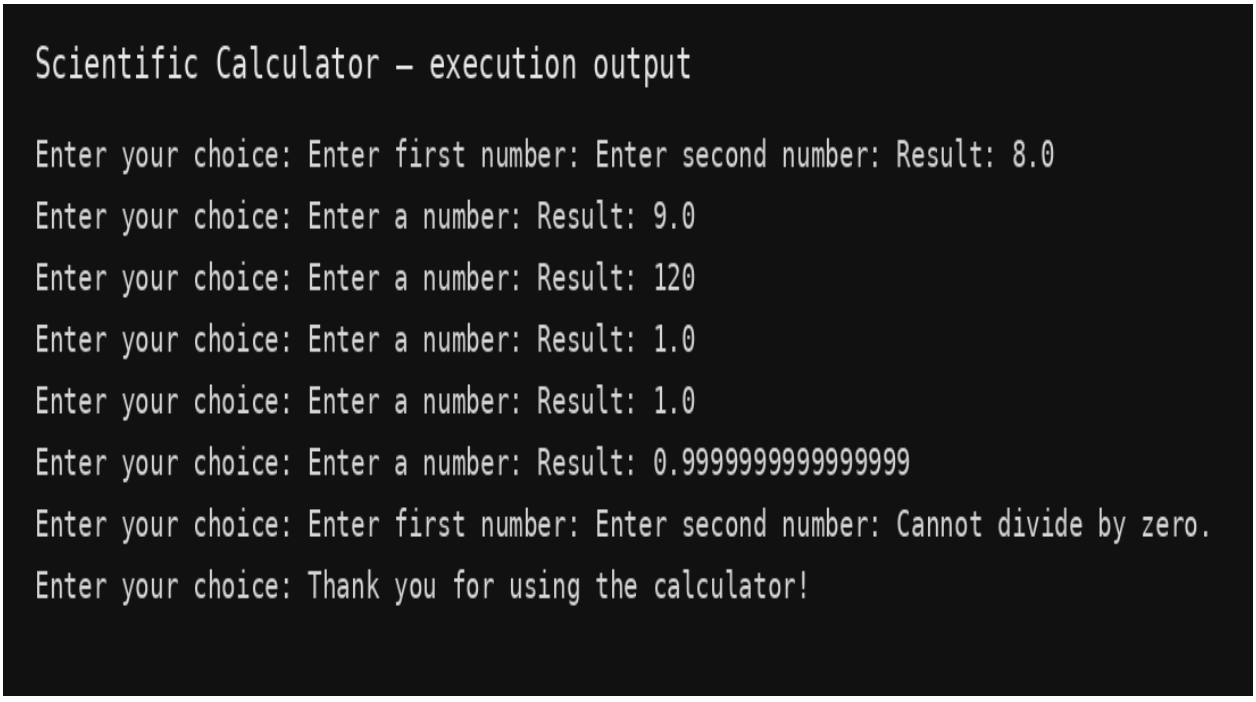
```
Scientific_Calculator_VITyarthi/
■■■ src/
■   ■■■ main.py
■   ■■■ calculator.py
■   ■■■ operations.py
■   ■■■ scientific.py
■   ■■■ validators.py
■   ■■■ menu.py
■   ■■■ ui.py
■■■ tests/test_calculator.py
■■■ docs/diagrams/
■■■ docs/screenshots/
■■■ README.md
■■■ statement.md
■■■ requirements.txt
■■■ GITHUB_PUSH.md
```

11. Screenshots / Results

The screenshots were generated from actual execution of the implemented command-line application. The run demonstrates arithmetic, power, square root, factorial, trigonometric functions, and the division-by-zero safeguard.



Working calculator execution



Error handling result

12. Testing Approach

The project includes six automated tests using pytest. The tests are focused on the calculation layer so they do not depend on interactive console input.

- Basic arithmetic: verifies addition, subtraction, multiplication, division, and power.
- Square and square root: verifies correct values.
- Factorial: verifies a valid factorial and rejection of a negative input.

- Trigonometry: verifies degree-based sine, cosine, and tangent values.
- Division by zero: verifies that a ZeroDivisionError is raised.
- Choice dispatch: verifies that menu choices reach the correct operation.

Execution result: 6 tests passed successfully using python -m pytest -q.

13. Challenges Faced

- Keeping the original menu-driven behavior while splitting the code into the required modular structure.
- Handling invalid numeric input without terminating the application.
- Handling mathematical domain errors such as square root of a negative number and factorial of a negative integer.
- Handling division by zero cleanly.
- Keeping trigonometric input intuitive by accepting degrees while Python math functions use radians internally.
- Creating documentation and diagrams that describe the modular implementation clearly.

14. Learnings & Key Takeaways

- Functions with one responsibility are easier to test and reuse.
- Python modules can separate user-interface code from business logic.
- Exception handling improves reliability in interactive programs.
- The standard math library provides reliable implementations of common scientific functions.
- Automated tests provide quick evidence that core calculations behave as expected.
- Clear architecture and diagrams make a small program easier to explain and maintain.

15. Future Enhancements

- Add logarithmic, exponential, percentage, reciprocal, and absolute-value operations.
- Add a calculation history feature.
- Add memory functions such as M+, M-, MR, and MC.
- Add a graphical user interface using Tkinter.
- Add keyboard shortcuts and expression-based input.
- Add configurable precision and formatted scientific notation.
- Add export of calculation history to a text or CSV file.

16. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (provided project brief).
- Python documentation – Python language, functions, modules, exceptions, and standard library concepts.
- Python math module documentation – square root, factorial, angle conversion, and trigonometric functions.
- pytest documentation – automated testing framework used for the project.

17. Submission Checklist

- README.md includes project title, overview, features, technologies, run instructions, and testing instructions.
- statement.md includes problem statement, scope, target users, and high-level features.
- Source code contains more than five meaningful modules/files.
- System architecture, workflow, use case, class/component, sequence, code-flow, and storage diagrams are included.
- Automated tests are included and pass successfully.
- Project report includes required sections and execution screenshots.
- Git configuration and GitHub push instructions are included.

Conclusion: The Scientific Calculator satisfies the supplied VITyarthi project structure by providing a meaningful Python application, modular implementation, documented requirements, design diagrams, testing evidence, and a submission-ready GitHub project structure.